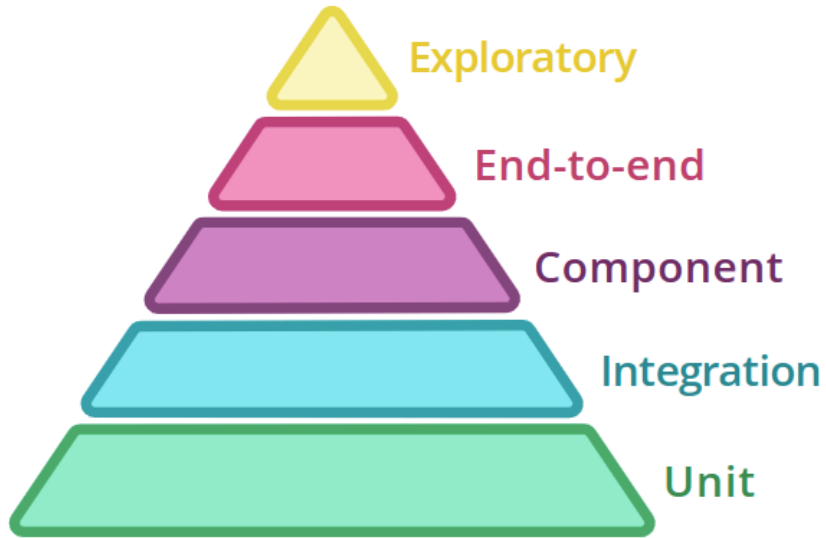


Arquitectura de Microservicios con Contenedores

- Ing. Federico Castañeda
- Ing. Carlos Maceira
- Ing. Franco Rupnik

Testing

Testing: Pirámide de Mike Cohn

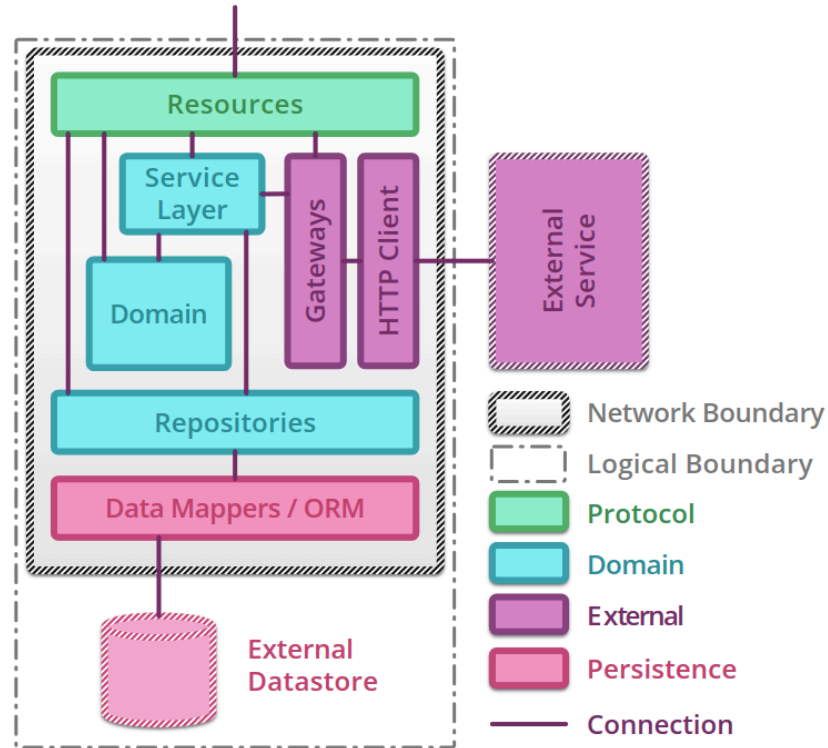


Al subir, **todas las dimensiones se mueven juntas**: más scope, más frágil, más lento, más caro de escribir, menos cantidad, más manual.

Lo que la pirámide no cubre: solo habla de tests automatizados de desarrollo. Quedan afuera el testing exploratorio, la usabilidad, la performance y la seguridad.

La mayoría de los equipos hace **solo la base** – y después se sorprende cuando producción explota por algo que ni miraron: se cayó bajo carga, o funcionaba pero nadie entendía cómo usarlo.

Estructura de microservicio



Fuente: <https://martinfowler.com/articles/microservice-testing/>

Unit Test

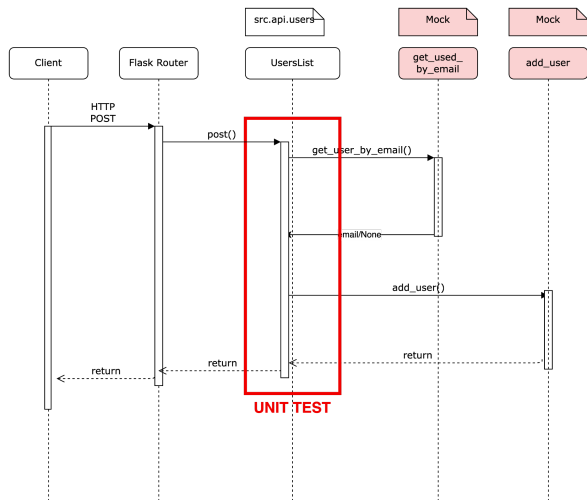
Orientado a evaluar una única función o método. Se desarrollan como parte del código y tienen como objetivo validar todos esos controles inherentes a la función.

- Valida correctamente los parámetros de ingreso.
- Dada una serie de parámetros particulares, verifica que la respuesta sea la esperada.
- Debe existir un test por cada "caso de uso" de la función
- Dan rápido feedback.
- Se realiza en las primeras etapas del pipeline.
- Primera barrera de contención para que el código no llegue al branch main sin haber sido testeado.

!! Si la escritura del test no es sencilla, entonces se debería dividir el módulo

Esa señal llegaba porque escribir ocho mocks dolía. Hoy no duele: hay que ir a buscarla.

Unit Test: Mocking



Solitary – todo lo de afuera mockeado. Si falla, sabés exactamente qué componente es.

Sociable – usa algunas dependencias reales. Te dice que las piezas encajan, pero cuando falla hay que investigar cuál de las cinco se rompió.

El vocabulario que vas a encontrar en cualquier framework:

- **Stub** – devuelve una respuesta fija. No verifica nada. *"Cuando te pidan la colección, devolvé estas tres figuritas."*
- **Mock** – además verifica que la interacción ocurrió. *"Comprobá que se llamó a guardar () exactamente una vez."*
- **Fake** – implementación real pero simplificada. Una base en memoria.

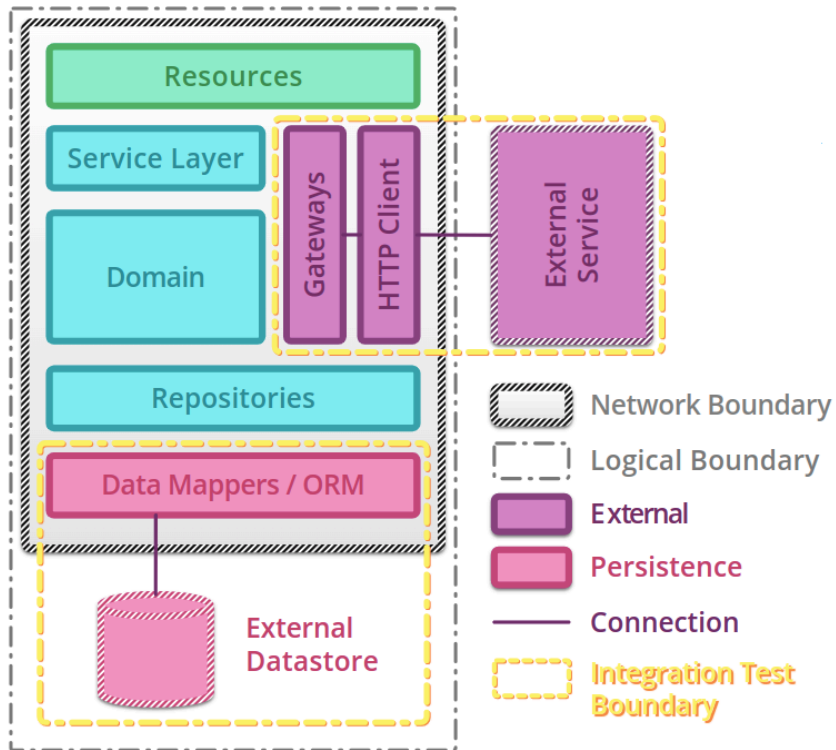
Mockear de más te da tests que pasan siempre y no prueban nada: terminás testeando tus propios mocks. Es el problema que resuelve **Contract Testing**.

Integration Test

Se focaliza en la integración entre servicio. Se realiza sobre la menor cantidad de servicios posible, idealmente, sólo dos.

- Busca verificar que las configuraciones necesarias para la integración están bien definidas.
- Evalúa ida y vuelta, requerimientos y respuesta.
- Implementan mocks y stubs para simular el resto de los componentes no evaluados.

Integration Test: los dos límites



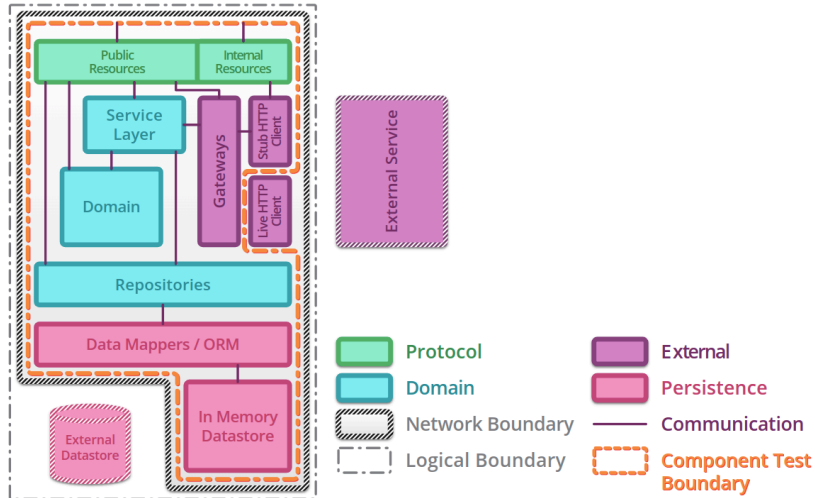
Integración con servicios externos

- Headers HTTP
- Request/Response
- Timeouts
- Circuit breakers

Integración con persistencia

- Mapeo ORM a BDD
- Transaccionalidad
- Timeouts
- Configuración de conexión

Component vs Functional Test



Component define un **alcance**: el servicio entero, aislado. Base efímera, vecinos stubbeados con WireMock.

Se opone a unit / integration / e2e.

Functional define una **intención**: que haga lo que el requerimiento pide. Se opone a lo no funcional – performance, seguridad, usabilidad.

Valida en tres niveles:

- **Sintaxis:** ¿el campo es un número? Lo agarra el framework.
- **Semántica:** ¿tiene sentido en el dominio? Una figurita n°900 en un álbum de 700.
- **Lógica:** ¿tiene sentido en este estado? Aceptar una oferta que ya fue aceptada.

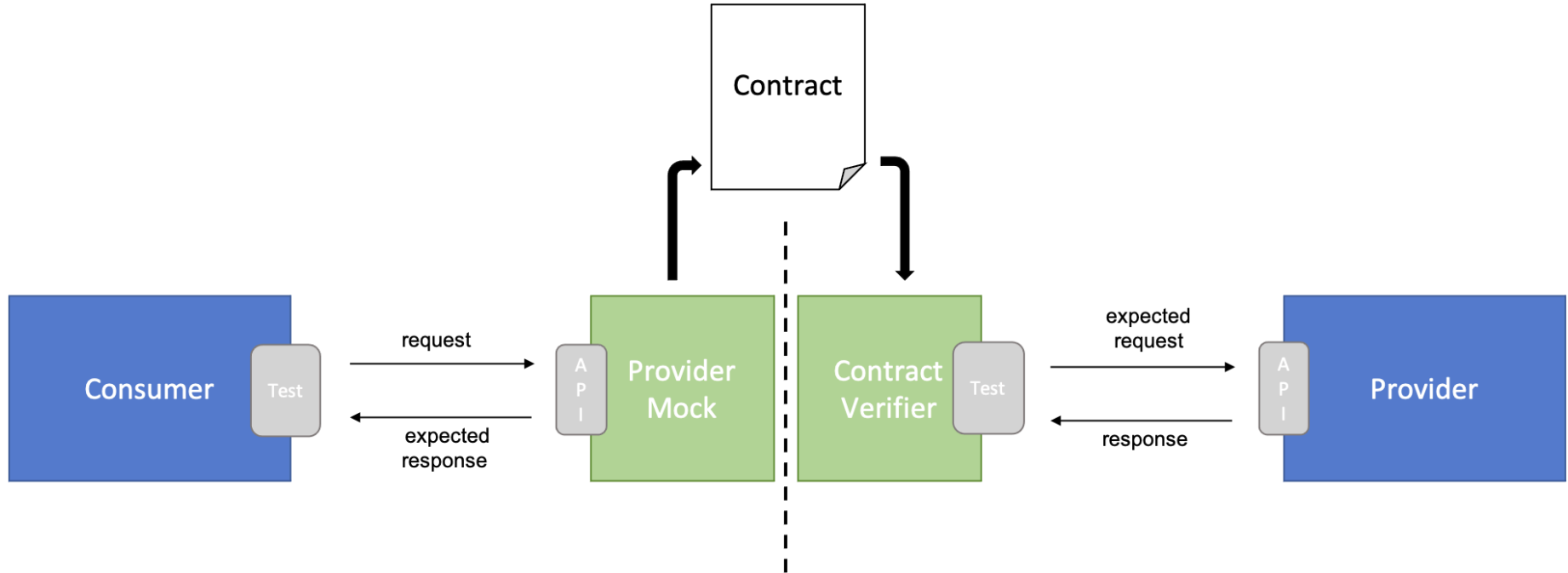
No son dos escalones: son dos preguntas distintas.

Contract Testing

Se testean las interfaces entre servicios bajo un contrato. Se especifica un protocolo y se define qué tipos de mensajes viajan por esa interfaz.

- Permite la validación del consumidor y del proveedor por separado.
- Normalmente mantiene un repositorio único y común de contratos.
- La simulación del consumidor o del proveedor permite simplificar el ambiente de pruebas.
- Son fundamentales en la implementación de Consumer-Driven Contracts (CDC).
- Son automatizables y suelen realizarse antes del deployment.

Contract Testing



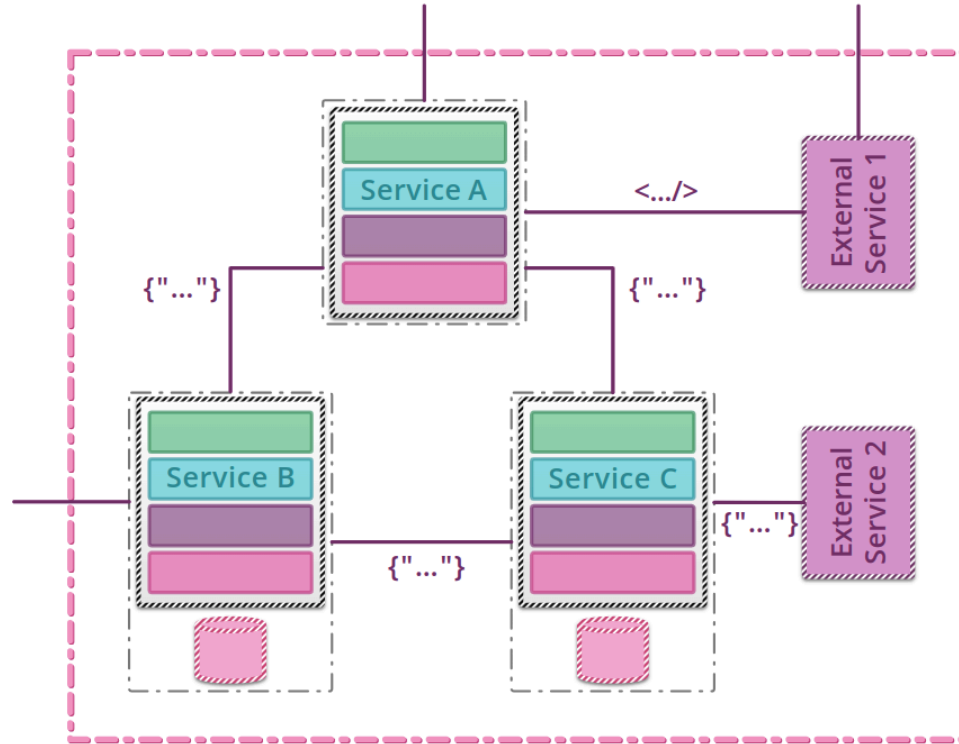
Fuente: <https://microsoft.github.io/code-with-engineering-playbook/automated-testing>

End-to-End testing

Implementan toda o gran parte de la infraestructura para producción de modo que emular todas las integraciones entre servicios.

- Son muy costosos de implementar y no permiten demasiada concurrencia.
- Deben diseñarse según la perspectiva del usuario final. Testear historias.
- Debe considerar múltiples escenarios dentro de esas historias.
- Escribirlos se abarató; **mantenerlos no** – y se rompen a la velocidad a la que cambia el sistema.
- Requieren un trabajo previo de análisis de las acciones más comunes de los usuarios para elegir las historias que más podrían impactar en el uso de la aplicación.

End-to-End testing



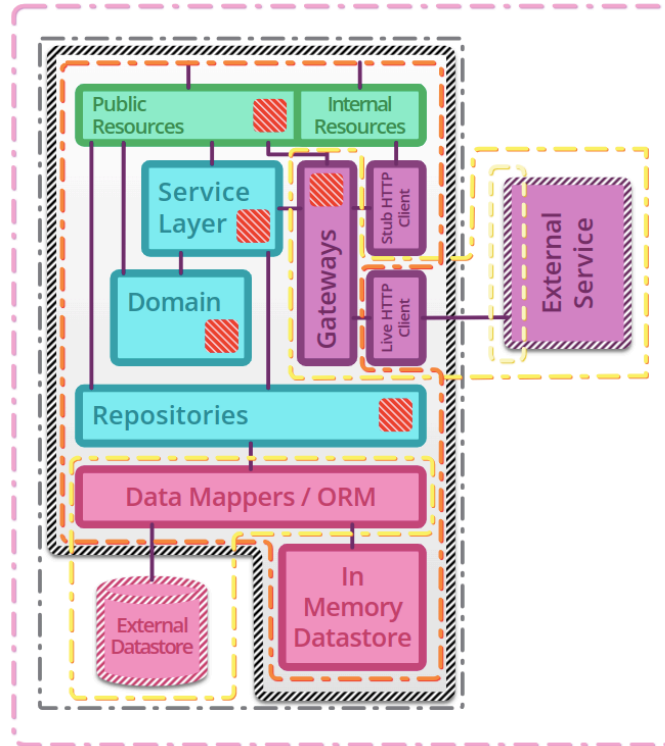
Fuente: <https://martinfowler.com/articles/microservice-testing/>

End-to-End testing

Recomendaciones

- Escribir la menor cantidad posible
- Enfocarse en historias de usuario
- Elegir el scope del test con criterio
- Hacer pruebas independientes de datos
- Mejorar monitoreo en producción

Resumen



Fuente: <https://martinfowler.com/articles/microservice-testing/>

Four-Phase Test

La anatomía de cualquier test, sea unit o de integración.

Fase	Qué hace	La trampa
1. Arrange	Define datos y contexto. Unit: unas variables. Integración: un despliegue ad-hoc	Si es más largo que Act + Assert juntos, el test es demasiado grande
2. Act	Ejecuta la acción sobre el ambiente	Debería ser una línea. Cinco pasos = probás un flujo, no una acción
3. Assert	Evalúa el resultado contra el esperado	Un assert por concepto. Y el peor de todos: el test sin assert
4. Clean-up	Destruye o resetea el ambiente para el test siguiente	La que todos se saltean, y la causa #1 de tests que "fallan a veces"

Los tests deben reutilizar el ambiente y **no ser destructivos entre ellos**: si B necesita que A haya corrido antes, no tenés dos tests.

Given-When-Then

Es **Arrange-Act-Assert** contado en lenguaje de **negocio**. Misma estructura, otro lector.

- **Given:** el estado del sistema antes de la acción.
- **When:** la acción en sí.
- **Then:** el estado que esperamos después.

Por eso vive en el cuadrante **Q2**: business facing. Un product owner lo puede leer.

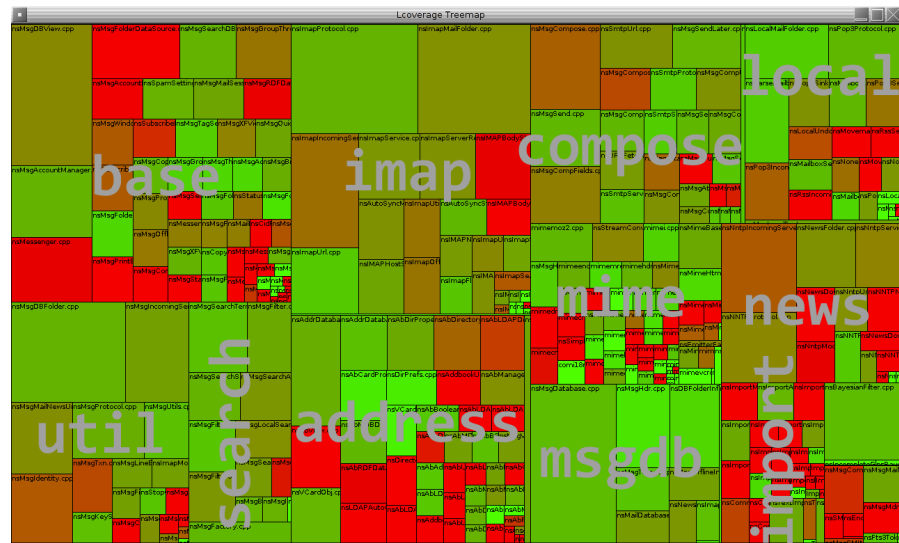
```
def test_ping(test_app):  
    # Given  
    client = test_app.test_client()  
  
    # When  
    resp = client.get('/ping')  
    data = json.loads(resp.data.decode())  
  
    # Then  
    assert resp.status_code == 200  
    assert 'pong' in data['message']  
    assert 'success' in data['status']
```

Cobertura

A la hora de evaluar la calidad de los tests realizados, se suele medir por medio del concepto de cobertura del código (code coverage).

Se define la cobertura como cuánto código, de todo el existente, fue ejecutado durante las pruebas.

Obviamente, si hay código que no fue ejecutado, entonces no fue testeado.



Cobertura

Coverage report: 99%

coverage.py v7.2.2, created at 2023-03-25 15:30 +0000

Module	statements	missing	excluded	branches	partial	coverage
src/__init__.py	17	1	0	2	0	95%
src/api/__init__.py	0	0	0	0	0	100%
src/api/models.py	16	0	0	2	0	100%
src/api/ping.py	8	0	0	0	0	100%
src/api/users.py	34	0	0	10	0	100%
src/config.py	12	0	0	0	0	100%
Total	87	1	0	14	0	99%

coverage.py v7.2.2, created at 2023-03-25 15:30 +0000

Coverage for src/__init__.py: 95%

17 statements 16 run 1 missing 0 excluded 0 partial

« prev » index » next coverage.py v7.2.2, created at 2023-03-25 15:30 +0000

```

1 import os
2
3 from flask import Flask # new
4 from flask_sqlalchemy import SQLAlchemy
5
6
7
8 # instantiate the db
9 db = SQLAlchemy()
10
11 def create_app(script_info=None):
12
13     # instantiate the app
14     app = Flask(__name__)
15
16     # set config
17     app_settings = os.getenv('APP_SETTINGS')
18     app.config.from_object(app_settings)
19
20     # set up extensions
21     db.init_app(app)
22
23     # register blueprints
24     from src.api.ping import ping_blueprint
25     app.register_blueprint(ping_blueprint)
26     from src.api.users import users_blueprint
27     app.register_blueprint(users_blueprint)
28
29     # shell context for flask cli
30     @app.shell_context_processor
31     def ctx():
32         return {'app': app, 'db': db}
33
34     return app
35
36

```

IMPORTANTE: Tener una cobertura del 100% de código no asegura que los tests estén cubriendo una evaluación efectiva del código. Testear al 100% no significa testear las cosas correctas.

Y generar tests que ejecuten todo el código hoy es trivial: **como número vale menos que nunca; como detector de huecos sigue sirviendo.**

Linting y Testing de Seguridad

Cuatro formas de revisar el código sin correr un test. Cambia **qué miran** y **cuándo**.

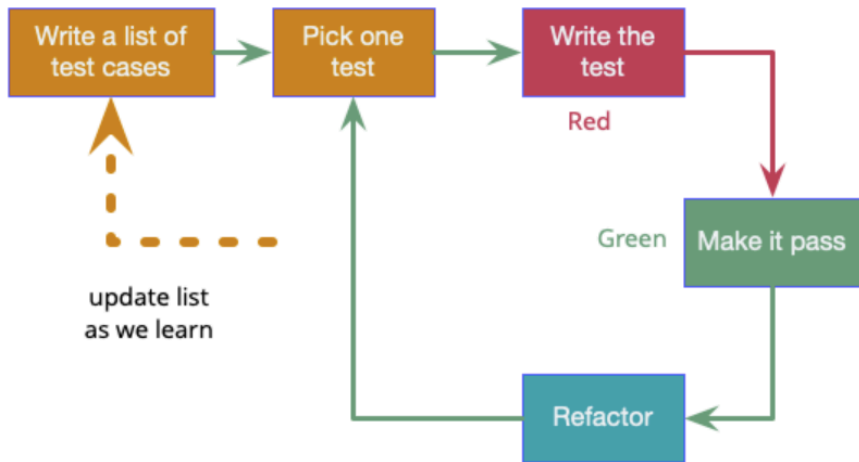
	Qué busca	Cuándo	Herramienta
Linter	Cómo está escrito: convenciones del equipo, patrones peligrosos	Al escribir, en el IDE	ESLint, Checkstyle, Hadolint
SAST Static Application Security Testing	Vulnerabilidades en tu código: SQL injection, XSS, input sin validar	En el commit	SonarQube, Semgrep
SCA Software Composition Analysis	Vulnerabilidades en tus dependencias	En el build	Trivy
DAST Dynamic Application Security Testing	La app corriendo, atacándola desde afuera	Ambiente de prueba	OWASP ZAP

SCA es el que más rinde: el 90% del código que deployás no lo escribiste vos. Log4Shell fue exactamente eso.

Ninguno verifica que el código haga **lo correcto** – eso es el test.

También vas a ver **IAST** (instrumenta la app en desarrollo) y **RASP** que no es testing: es defensa en producción

Test-Then-Code



Escribís el test que falla → lo mínimo para que pase
→ limpiás.

El test es el **primer cliente** de tu código: te obliga a definir cómo se usa antes de cómo se hace.

Qué cambió

- **Hoy los tests son gratis.** La objeción de siempre a TDD era el costo de escribirlos primero. Esa objeción ya no existe.
- Lo que **no** es gratis: saber qué tiene que hacer. El test vale por el requerimiento que codifica, no por existir.
- **Nunca dejes que el modelo ajuste el test al código.** Si escribe los dos, el test se acomoda a la implementación y pasa siempre.

Función NextDate

Historia

El sistema debe calcular el día siguiente a una fecha ingresada.

User Stories

#	
1	The program compiles.
2	A day can be input and displayed.
3	An input month can be displayed.
4	An input year can be displayed.
5	A day below minimum can be detected.
6	A day above maximum can be detected.
7	A month below minimum can be detected.
8	A month above maximum can be detected.
9	A year below minimum can be detected.
10	A year above maximum can be detected.

Función NextDate

#	
11	Day = 31 in a 30 day month.
12	Day = 30 in February (note: also need Day = 31 in February).
13	Day = 29 in February in a common year.
14	Day = 29 in February in a leap year.
15	A year divisible by 4 is a leap year (except for century years).
16	A year not divisible by 4 is a common year.
17	A century year not divisible by 400 is a common year.
18	A century year divisible by 400 is a leap year.

#	
19	Increment a non-last day of a month.
20	Increment the last day of a 30-day month.
21	Increment the last day of a 31-day month.
22	Increment December 31.
23	Increment February 28 in a common year.
24	Increment February 28 in a leap year.
25	Increment February 29 in a leap year.

Fin !!